

Bit Manipulation — Complete Revision Notes

Source Lecture Notes: :contentReference[oaicite:0]{index=0}

What is Bit Manipulation?

Bit Manipulation means performing operations directly on binary digits (0 and 1) using bitwise operators.

Computers internally understand only binary numbers.

Example:

```
Decimal 5 = Binary 0101
```

Bit manipulation helps in:

- Reducing Time Complexity
 - Optimizing memory usage
 - Solving advanced DSA questions
 - Competitive programming
 - Low-level programming
 - Operating systems
 - Networking
-

Binary Number System

Binary has only:

```
0 and 1
```

Where:

Bit	Meaning
1	Current/Electricity Present
0	No Current

Bit Positions

Positions are counted from RIGHT to LEFT starting from 0.

Example:

```
Binary: 0101
```

```
Position: 3 2 1 0
```

```
Bits:      0 1 0 1
```

Important Bitwise Operators

Operator	Meaning
&	AND
,	,
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

MOST IMPORTANT QUESTIONS

What is Left Shift Operator (<<) ?

Definition

Left shift moves all bits towards LEFT.

Syntax:

```
number << positions
```

Example

```
3 << 1
```

Binary:

```
3 = 0011
```

```
0011 << 1
```

```
=
```

```
0110
```

Result:

```
6
```

Why?

Every left shift multiplies the number by 2.

```
n << 1 = n × 2
```

```
n << 2 = n × 4
```

Core Logic

Bits move left and 0 enters from right side.

What is Right Shift Operator (>>) ?

Definition

Right shift moves all bits towards RIGHT.

Syntax:

```
number >> positions
```

Example

```
4 >> 1
```

Binary:

```
0100 >> 1
=
0010
```

Result:

```
2
```

Why?

Every right shift divides number by 2.

```
n >> 1 = n / 2
n >> 2 = n / 4
```

Core Logic

Bits move right and leftmost position fills depending on sign.

What is Bit Masking?

Definition

Bit Masking means using another binary number (mask) to manipulate specific bits.

General Process

Step 1 → Create Bit Mask

```
1 << pos
```

Step 2 → Perform Operation

Using:

- AND
- OR

- XOR
- NOT

Why Bit Masking is Powerful?

It allows changing/checking only one bit without affecting others.

What is GET BIT Operation?

Purpose

Check whether a specific bit is 0 or 1.

Formula

```
bitMask = 1 << pos  
n & bitMask
```

Logic

- If result = 0 → bit is OFF
 - If result ≠ 0 → bit is ON
-

Example

```
n = 0101  
pos = 2
```

Create Mask

```
1 << 2 = 0100
```

AND Operation

```
0101  
0100
```

```
----  
0100
```

Result is non-zero.

Therefore:

```
Bit is 1
```

Java Code

```
int n = 5;  
int pos = 2;  
  
int bitMask = 1 << pos;  
  
if((n & bitMask) == 0) {  
    System.out.println("Bit was 0");  
} else {  
    System.out.println("Bit was 1");  
}
```

What is SET BIT Operation?

Purpose

Convert a specific bit into 1.

Formula

```
n | (1 << pos)
```

Why OR Operator?

Because:

```
1 | anything = 1
```

So target bit always becomes 1.

Example

```
n = 0101  
pos = 1
```

Mask

```
0010
```

OR Operation

```
0101  
0010  
----  
0111
```

Result:

```
7
```

Java Code

```
int n = 5;  
int pos = 1;  
  
int bitMask = 1 << pos;  
  
int newNumber = n | bitMask;  
  
System.out.println(newNumber);
```

What is CLEAR BIT Operation?

Purpose

Convert a specific bit into 0.

Formula

```
n & ~(1 << pos)
```

Why NOT + AND ?

NOT (~)

Converts:

```
1 → 0  
0 → 1
```

AND (&)

Because:

```
0 & anything = 0
```

So target bit becomes 0.

Example

```
n = 0101  
pos = 2
```

Mask

```
0100
```

NOT Mask

```
1011
```

AND

```
0101
1011
----
0001
```

Result:

```
1
```

Java Code

```
int n = 5;
int pos = 2;

int bitMask = 1 << pos;

int newBitMask = ~bitMask;

int newNumber = newBitMask & n;

System.out.println(newNumber);
```

What is UPDATE BIT Operation?

Purpose

Update bit to either:

- 1 OR
- 0

depending on user choice.

Two Cases in UPDATE

Operation	Meaning
SET	Make bit = 1

Operation	Meaning
CLEAR	Make bit = 0

Logic

If oper = 1

Perform SET operation.

```
n | bitMask
```

If oper = 0

Perform CLEAR operation.

```
(~bitMask) & n
```

Full Java Code

```
import java.util.*;

public class UpdateBit {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int oper = sc.nextInt();

        int n = 5;
        int pos = 2;

        int bitMask = 1 << pos;

        if(oper == 1) {

            // SET
            int newNumber = bitMask | n;
            System.out.println(newNumber);

        } else {
```

```
        // CLEAR
        int newBitMask = ~(bitMask);
        int newNumber = newBitMask & n;

        System.out.println(newNumber);
    }

    sc.close();
}
}
```

MOST IMPORTANT LOGICAL CONCEPTS

Why Do We Use $1 \ll \text{pos}$?

It creates a mask containing only ONE active bit.

Example:

```
1 << 2
```

becomes:

```
0100
```

Only position 2 contains 1.

This allows us to manipulate only one bit.

Why Does AND Work in GET/CLEAR?

Because:

```
0 & anything = 0
1 & 1 = 1
```

AND helps isolate or remove bits.

Why Does OR Work in SET?

Because:

```
1 | anything = 1
```

So OR guarantees the bit becomes 1.

Why is Logic More Important Than Memorizing?

In interviews and coding tests:

- Questions may be modified
- Conditions may change
- Direct formulas may not work

If logic is understood:

- New questions become easy
 - Custom operations can be built
-

IMPORTANT QUESTIONS

Why Does Clearing Position 2 in 0101 Give 0001?

Initial:

```
0101
```

Position 2 contains:

```
1
```

After CLEAR:

```
0001
```

which is decimal:

```
1
```

Why Are Outputs Printed in Decimal?

Java prints integers in decimal by default.

Example:

```
0111 -> 7
0001 -> 1
```

To print binary:

```
Integer.toBinaryString(number)
```

Quick Revision Table

Operation	Formula
Get Bit	$n \& (1 \ll pos)$
Set Bit	$n 1$
Clear Bit	$n \& (\sim(1 \ll pos))$
Update Bit	SET or CLEAR

Full Lecture Summary

This lecture covered:

- Binary Number System
- Left Shift Operator
- Right Shift Operator
- Bit Masking
- Get Bit
- Set Bit
- Clear Bit

- Update Bit
- Logical understanding behind all operations
- Importance of understanding logic instead of memorization

Main idea of the lecture:

Bit Manipulation allows efficient control over individual bits using bitwise operators and bit masks.

This topic is extremely important for:

- DSA
- Competitive Programming
- Advanced Coding Interviews
- Optimization Problems

Final Core Takeaway

Remember these 4 formulas:

```
// GET
n & (1 << pos)

// SET
n | (1 << pos)

// CLEAR
n & ~(1 << pos)

// UPDATE
Use SET or CLEAR depending on requirement
```

These are the foundation of complete Bit Manipulation.